

Step-by-Step Setup Guide (with explanations)

This walks through building the project from zero, in the order you'd actually do it. Each step says **what** you're doing, **why**, and **which technology is responsible for what**. Follow it in order — later steps depend on earlier ones.

Step 0: Install prerequisites

What: Install Node.js (LTS, v20+), Git, and Docker Desktop on your machine. Confirm with:

```
node -v
git --version
docker --version
```

Why: Node.js is the JavaScript runtime that runs Next.js both in development and in production — without it, nothing in this stack runs. Docker is what lets you run PostgreSQL locally in an isolated container instead of installing Postgres directly on your OS (avoids version conflicts, and mirrors production). Git is your version control; since you already have a GitHub repo, this connects your local machine to it.

Responsible for: Node.js = JS execution environment. Docker = isolated, reproducible environments for services like the database. Git = version history and collaboration.

Step 1: Scaffold the Next.js app

What:

```
npx create-next-app@latest your-app --typescript --tailwind --eslint --app --src-dir
cd your-app
```

Why: `create-next-app` is the official scaffolding tool — it sets up the build tooling, config files, and folder conventions so you don't hand-configure a bundler, TypeScript compiler, etc. yourself. The flags matter:

- `--typescript`: catches type errors (like calling a function with the wrong shape of data) before you even run the code, rather than discovering them as runtime crashes.
- `--tailwind`: a utility-first CSS approach — you style elements with classes like `flex items-center` directly in your markup instead of writing separate CSS files. Faster to iterate with as a junior dev since you're not context-switching between files.
- `--app`: uses the App Router (Next.js's current architecture), which is what lets you colocate frontend pages and backend API routes in the same folder tree — this is the piece that makes "Next.js full-stack" possible.
- `--src-dir`: puts your code in `src/` instead of the project root, keeping app code separate from config files.

Responsible for: Next.js is responsible for both **rendering your React UI** (pages, components) and **serving your API** (Route Handlers) — one framework, one dev server, one deploy artifact.

Step 2: Connect to your existing GitHub repo

What:

```
git remote add origin <your-repo-url>
git add .
git commit -m "Initial commit: scaffolded Next.js app"
git push -u origin main
```

Why: You want the scaffolded project tracked in version control from the very first commit, so every change from here on is reviewable and reversible. Doing this before you start adding business logic means you always have a clean, buildable "empty state" to fall back to.

Responsible for: GitHub = remote source of truth for the codebase, and later, the trigger point for your CI/CD pipeline (Step 10).

Step 3: Run PostgreSQL locally with Docker Compose

What: Create `docker/docker-compose.yml`:

```
services:
  postgres:
    image: postgres:16
    environment:
      POSTGRES_USER: dev
      POSTGRES_PASSWORD: dev
      POSTGRES_DB: appdb
    ports:
      - "5432:5432"
    volumes:
      - pgdata:/var/lib/postgresql/data
volumes:
  pgdata:
```

Then run:

```
docker compose -f docker/docker-compose.yml up -d
```

Why: Your app needs a real database to develop against, but you don't want every teammate manually installing and configuring Postgres on their own OS (different versions cause subtle bugs). Docker Compose describes the database as code — anyone on the team runs one command and gets an identical database. The `volumes` entry means your data survives container restarts (you don't lose your test data every time you stop Docker).

Responsible for: PostgreSQL = the actual data store (users, records, everything persistent). Docker = making that data store reproducible and disposable in development.

Step 4: Add Prisma (the ORM)

What:

```
npm install prisma @prisma/client
npx prisma init
```

This creates `prisma/schema.prisma`. Set `DATABASE_URL="postgresql://dev:dev@localhost:5432/appdb"` in `.env.local`. Define a first model:

```
model User {
  id      String  @id @default(cuid())
  email   String  @unique
  name    String?
  role    Role    @default(USER)
  createdAt DateTime @default(now())
}

enum Role {
  USER
  ADMIN
}
```

Then run:

```
npx prisma migrate dev --name init
```

Why: Without an ORM, you'd write raw SQL strings in your code — easy to make typos in, hard to keep in sync as your schema changes, and vulnerable to injection bugs if done carelessly. Prisma lets you define your data model once, in a readable schema file, and generates a fully-typed client so your editor autocompletes `prisma.user.findMany()` and TypeScript yells at you if you typo a field name. `migrate dev` also keeps a history of every schema change as a file, so your team and your production database evolve in lockstep instead of drifting apart.

The `role` field here is the foundation for Step 6 (role-based middleware) — you're deciding at the data layer that every user has a role, before you ever write the code that checks it.

Responsible for: PostgreSQL = storage. Prisma = the typed interface your app code uses to read/write that storage, plus schema migration history.

Step 5: Set up Auth.js (NextAuth)

What:

```
npm install next-auth@beta @auth/prisma-adapter
```

Generate a secret:

```
npx auth secret
```

Create `src/lib/auth.ts`:

```
import NextAuth from "next-auth";
import { PrismaAdapter } from "@auth/prisma-adapter";
import { prisma } from "@lib/db";
import Google from "next-auth/providers/google";

export const { handlers, auth, signIn, signOut } = NextAuth({
  adapter: PrismaAdapter(prisma),
  providers: [
    Google({
      clientId: process.env.AUTH_GOOGLE_ID,
      clientSecret: process.env.AUTH_GOOGLE_SECRET,
    }),
  ],
  callbacks: {
    async session({ session, user }) {
      if (session.user) session.user.role = user.role;
      return session;
    },
  },
});
```

Wire it into a route handler at `src/app/api/auth/[...nextauth]/route.ts`:

```
import { handlers } from "@lib/auth";
export const { GET, POST } = handlers;
```

Why: Authentication — securely handling passwords/OAuth tokens, sessions, cookies — is easy to get *subtly* wrong (session fixation, insecure cookie flags, CSRF) if you build it yourself. Auth.js is a maintained library that handles those security details for you and plugs directly into the Prisma adapter, meaning your `User` table (Step 4) becomes the source of truth for who's logged in, with no separate auth database to keep in sync.

The `session` callback above is the specific line that takes the `role` field you added to the `User` model and attaches it to every logged-in user's session object — this is what Step 6 reads from.

Responsible for: Auth.js = login flow, session/cookie management, OAuth provider integration. Prisma adapter = the glue that persists sessions/accounts into your Postgres database via Prisma.

Step 6: Add role-based middleware

What: Create `src/middleware.ts`:

```
import { auth } from "@lib/auth";
import { NextResponse } from "next/server";

export default auth((req) => {
  const isAdminRoute = req.nextUrl.pathname.startsWith("/admin");
  const role = req.auth?.user?.role;

  if (isAdminRoute && role !== "ADMIN") {
    return NextResponse.redirect(new URL("/", req.nextUrl));
  }
});

export const config = {
  matcher: ["/admin/:path*", "/dashboard/:path*"],
};
```

Why: Middleware runs *before* a request reaches your page or API route. That means you check "is this user allowed here?" in one central place, rather than repeating an `if` check at the top of every admin page and every admin API route (which is easy to forget on a new route later). The `matcher` config scopes this check to only the paths that need it, so public pages aren't slowed down by an unnecessary auth check.

Responsible for: Next.js middleware = the enforcement point. Auth.js session (from Step 5) = the source of the `role` data the middleware is checking against.

Step 7: Automated emails (Resend or Nodemailer + SES)

What (Resend path — recommended to start with, simpler):

```
npm install resend react-email @react-email/components
```

Create `src/lib/email/client.ts`:

```
import { Resend } from "resend";
export const resend = new Resend(process.env.RESEND_API_KEY);
```

Send an email:

```
await resend.emails.send({
  from: "you@yourdomain.com",
  to: user.email,
  subject: "Welcome!",
  react: WelcomeEmail({ name: user.name }),
});
```

Why: Sending email reliably (avoiding spam folders, handling bounces, managing deliverability reputation) is a whole discipline on its own — you don't want to run your own mail server. Resend (or SES) handles the actual delivery infrastructure; your code just calls an API. Wrapping it in `src/lib/email/client.ts` means

the rest of your app calls one function without knowing or caring which provider is behind it — if you switch to SES later (e.g. because your sponsor wants everything in AWS), you change this one file, not every call site.

Responsible for: Resend/SES = actual email delivery and deliverability. Your `email/client.ts` wrapper = a stable interface so the rest of the app doesn't depend on a specific vendor.

Step 8: Containerize the app for deployment

What: Add to `next.config.js`:

```
module.exports = { output: "standalone" };
```

Create `docker/Dockerfile`:

```
FROM node:20-alpine AS deps
WORKDIR /app
COPY package.json package-lock.json ./
RUN npm ci

FROM node:20-alpine AS builder
WORKDIR /app
COPY --from=deps /app/node_modules ./node_modules
COPY . .
RUN npx prisma generate
RUN npm run build

FROM node:20-alpine AS runner
WORKDIR /app
ENV NODE_ENV=production
COPY --from=builder /app/.next/standalone ./
COPY --from=builder /app/.next/static ./next/static
COPY --from=builder /app/public ./public
EXPOSE 3000
CMD ["node", "server.js"]
```

Why: This is a **multi-stage build** — each `FROM` starts a fresh, minimal layer, and you only copy forward the specific output you need (installed deps, then the built app), rather than shipping your entire source tree, dev dependencies, and build cache to production. `output: "standalone"` tells Next.js to bundle only the files actually needed at runtime into one folder, which is what makes the final image small. A smaller image means faster deploys and a smaller attack surface.

Responsible for: Docker = packaging your app (code + runtime + dependencies) into one portable artifact that runs identically on your machine, a teammate's machine, and AWS.

Step 9: Deploy to AWS

What (high level, since exact steps depend on your AWS account setup):

1. Push the built image to **Amazon ECR** (a private Docker image registry).
2. Run it on **ECS Fargate** (serverless containers — AWS manages the underlying servers) or **App Runner** (even less configuration, good for a first deployment).
3. Store `DATABASE_URL`, `AUTH_SECRET`, `RESEND_API_KEY` etc. in **AWS Secrets Manager**, and reference them as environment variables in your task definition — never bake secrets into the Docker image.
4. Point your production `DATABASE_URL` at an **RDS PostgreSQL** instance rather than the local Docker Postgres from Step 3.

Why: ECS/Fargate or App Runner run your container without you managing the underlying EC2 servers yourself (patching, scaling). Secrets Manager exists because a secret baked into an image is baked into every layer's history — anyone who gets the image gets the secret, permanently. RDS gives you managed backups, automated failover, and patching for your production database, which you don't get from a database running in a plain Docker container.

Responsible for: ECR = image storage. ECS/App Runner = running the container. RDS = managed production database. Secrets Manager = secret storage/injection at runtime.

Step 10: CI/CD with GitHub Actions

What: Create `.github/workflows/deploy.yml` to run on every push to `main`:

```
name: Deploy
on:
  push:
    branches: [main]
jobs:
  build-and-deploy:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - name: Build and push to ECR
        run: |
          # docker build, tag, push steps here
      - name: Deploy to ECS
        run: |
          # aws ecs update-service --force-new-deployment
```

Why: Without this, deploying means someone manually building the Docker image and running AWS CLI commands from their own laptop — slow, error-prone, and undocumented. A GitHub Actions workflow means every merge to `main` automatically builds and deploys the same way, every time, and the deployment steps themselves are version-controlled and reviewable like any other code.

Responsible for: GitHub Actions = automation/orchestration of the build-test-deploy pipeline, triggered by your Git history (Step 2).

How it all fits together

You do this...	...because this technology needs it
Write React components in <code>app/</code>	Next.js renders your UI
Write Route Handlers in <code>app/api/</code>	Next.js also serves your backend — no separate server
Define models in <code>schema.prisma</code>	Prisma generates a typed client + migration history
Run <code>docker compose up</code>	Gives you a local Postgres matching production
Configure <code>auth.ts</code>	Auth.js manages login/session security for you
Add <code>role</code> to the session callback	Feeds your middleware's authorization check
Write <code>middleware.ts</code>	Enforces "who can see what" in one place
Wrap Resend/SES in <code>email/client.ts</code>	Keeps your app decoupled from the email vendor
Write a multi-stage <code>Dockerfile</code>	Packages the app identically for every environment
Push to ECR, run on ECS/App Runner	AWS runs your container without you managing servers
Write <code>deploy.yml</code>	Automates build → push → deploy on every merge

Follow the steps in this order and each one has what it needs from the step before it — nothing here requires jumping around.